

ELEG 55903: Programming for Power Electronics

Midterm

Tyler Cash

ELEG 4593

Midterm

2 April 2026

Abstract:

In this Midterm assignment, we were instructed to create a Serial to SPRAM Register Interface in VHDL, with an RX input and TX output. This project utilizes SPRAM, Serial, and Bus Master Interfaces to allow for updating internal registers using a serial interface. We also used the internal oscillator and PLL to generate the clk and reset.

Table of Contents:

Abstract:.....	2
List of Figures:.....	3
Overview:.....	3
Modules:.....	3
States:	3
User Interface:	4
Bus Master:.....	4
Bus Interface:.....	4
Testbench:.....	4
Conclusion:	5
Appendix A (Uart_Usr_Int):.....	6
Appendix B (Bus_Interface_Top):.....	8
Appendix C (Bus_Interface_TB):	8

List of Figures:

FIGURE 1. SERIAL TO REGISTER INTERFACE WAVEFORM.....	6
FIGURE 2. SERIAL TO REGISTER INTERFACE TERMINAL.....	6

Overview:

This midterm project involves implementing a Serial to SPRAM Register Interface in VHDL with the input RX and output TX. This project is similar to our last project, the Serial Echo Program, but this project is more comprehensive and enables packet-based communication between the Serial and SPRAM Register interfaces. Like the Echo Program, this project utilizes the PLL and internal oscillator to generate the clk and the reset, but this project also involved many other modules.

Modules:

For this assignment, there were quite a few modules and components required. We used a Bus Interface Top module to connect the RS232 User Interface, the Bus Master. In addition to those top modules, we also make sure of the Bus Interface Common, DSPRAM, PLL, and other generic components. This included many counters, reset blocks, and registers.

States:

This project required a much more extensive state sequence to control the data flow between receiving and transmitting. In total, there are 36 states, ensuring that the bus is shared between transmitting and receiving, and that the appropriate data is being transmitted or received based on the calculated checksum.

User Interface:

The UART User Interface is the main driver of this project, and where the large 36 state machine lives, coordinating the transmission of data between all the different components on the bus. This allows data to be written to the internal registers in the SPRAM using the serial interface and then write back output using the serial interface. The data is sent serially bit by bit and then is parsed into packets, using structured commands and checksum validation. The checksum is calculated by adding up all the bytes of the packet between the packet length and the checksum and subtracting the sum from 0xFF.

Bus Master:

The Bus Master oversees coordinating which component is talking on the bus. This also uses a state machine, containing 6 states, defining the behavior of which component or module can have access to the bus at a time. It controls all of the read and write operations with the SPRAM.

Bus Interface:

The Bus Interface Top module pairs with the Bus Master and the UART User Interface together, connecting the shared bus signals to enable controlled access. The Bus Interface Top module also generates the system clock, using the internal oscillator and PLL, as well as coordinating the reset for both modules.

Testbench:

The testbench starts with a reset, ensuring stable signals across all components, and then begins with the write command. Then the read command is sent to read the data from the registers. This testbench is robust to test all states for all of the state machines involved, ensuring that each case is tested and that we have achieved proper functionality. The waveform for the simulation is in Figure 1. I used the same read/write commands from the test bench when communicating serially with the board, using X-CTU, and that is shown in Figure 2.

Conclusion:

In conclusion, we have properly implemented the Serial to SPRAM Register Interface in VHDL. There were a lot of challenges with this project, it was the largest project thus far and had a ton of moving parts. When trying to debug, it wasn't always obvious where the issue was. I had forgotten to include certain signals in my sensitivity list for my UART User Interface, in the 36-state state machine, and it required an expert engineer to spot the error. With that said, I do feel like I learned a lot, especially when debugging, digging into the simulation, and looking at specific signals to see where the error was happening vs the expected behavior. I do expect that future problems will be much easier to spot and debug.

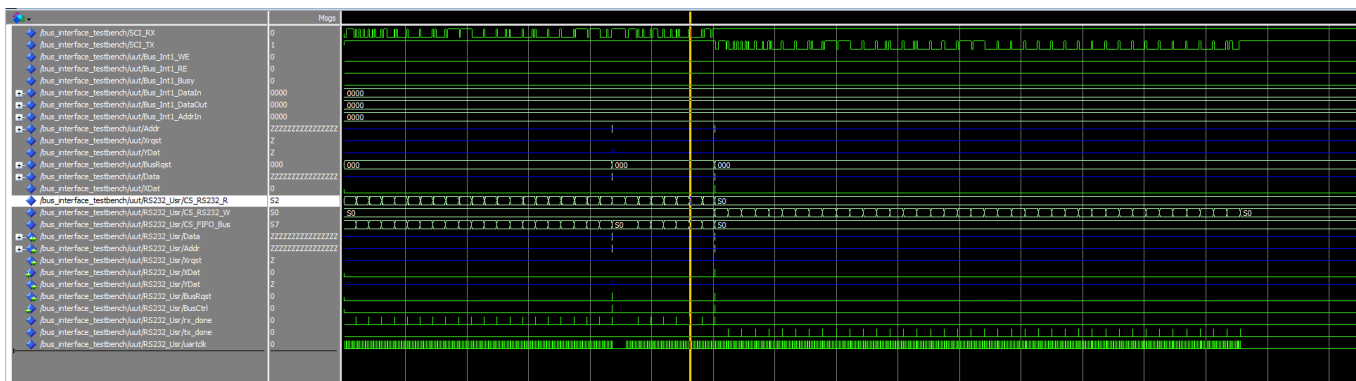


Figure 1. Serial to Register Interface Waveform

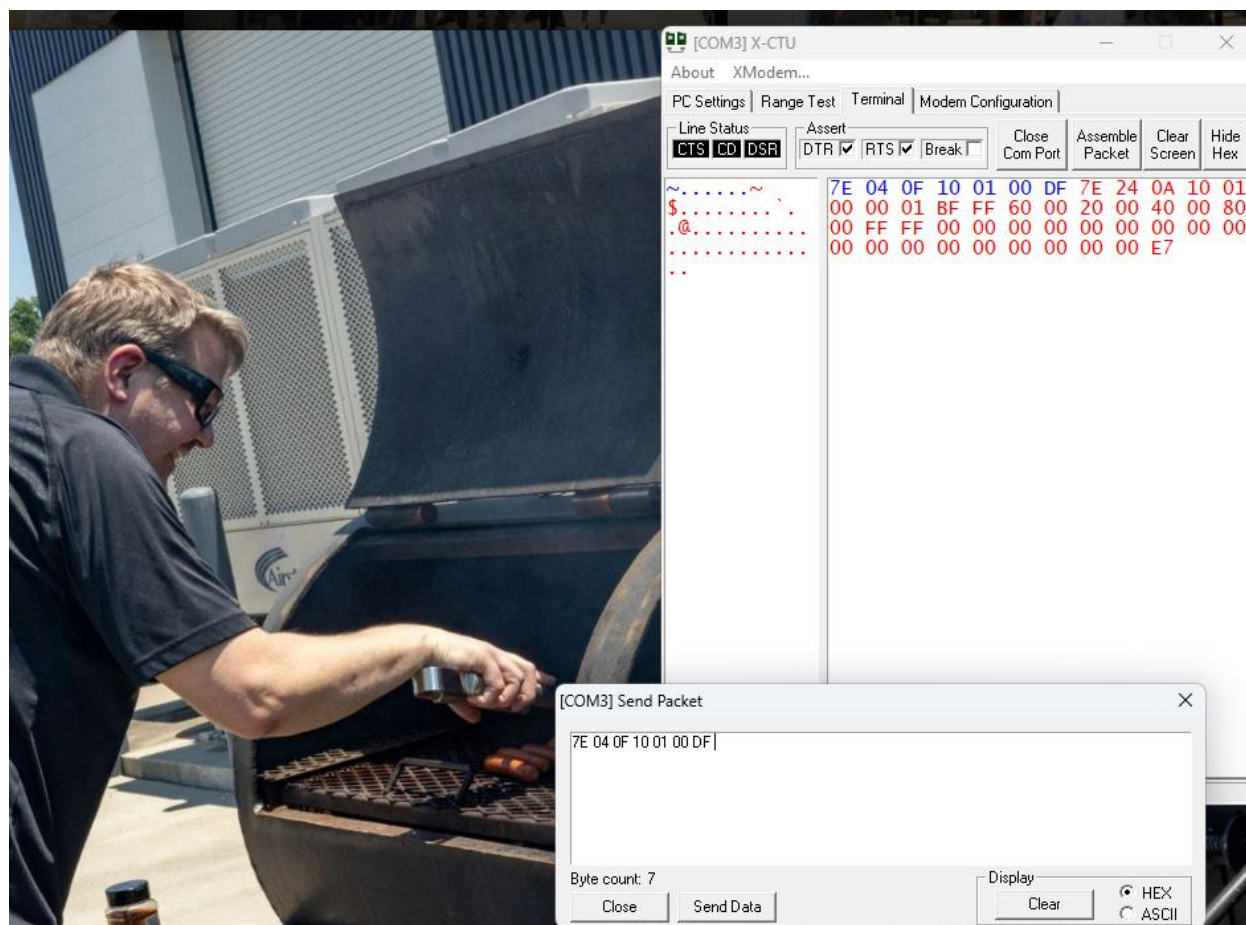


Figure 2. Serial to Register Interface Terminal

Appendix A (Uart Usr Int):

Library IEEE; use IEEE.std_logic_1164.all; use IEEE.std_logic_unsigned.all; use IEEE.numeric_std.all; use IEEE.std_logic_arith.all;

library work; use work.Bus_Interface_Common.all;

entity RS232_Usr_Int is

generic(Baud : integer := 9600; clk_in : integer := 25000000); Port (clk : in STD_LOGIC; rst : in STD_LOGIC; rs232_rcv : in STD_LOGIC; rs232_xmt : out STD_LOGIC; Data : INOUT std_logic_vector(15 downto 0); Addr : OUT std_logic_vector(15 downto 0); Xrqst : OUT std_logic; XDat : IN std_logic; YDat : OUT std_logic; BusRqst : OUT std_logic; BusCtrl : IN std_logic); end RS232_Usr_Int;

architecture Behavioral of RS232_Usr_Int is type state_type is (S0, S1, S2, S3, S4, S5, S6, S7, S8, S9, S10, S11, S12, S13, S14, S15, S16, S17, S18, S19, S20, S21, S22, S23, S24, S25, S26, S27, S28, S29, S30, S31, S32, S33, S34, S35, S36, S37, S8_wait); signal CS_RS232_R, NS_RS232_R, CS_RS232_W, NS_RS232_W, CS_FIFO_Bus, NS_FIFO_Bus: state_type; signal rx_done, tx_done :STD_LOGIC:= '0'; signal temp_rcv: STD_LOGIC_VECTOR(7 downto 0):= (others => '0'); signal i, j: STD_LOGIC_VECTOR (15 downto 0):= (others => '0'); signal uartclk : STD_LOGIC:= '0'; signal u: integer; signal rs232_rcv_s, rs232_rcv_t: STD_LOGIC:= '1'; signal txbuff:STD_LOGIC_VECTOR(9 downto 0):= (others => '1'); --buff used to transmit 1 byte wih start and stop bits

--Declare Signals for FIFO Serial Read

signal STD_FIFO_R_WriteEn, STD_FIFO_R_ReadEn: STD_LOGIC:= '0';
signal STD_FIFO_R_DataIn, STD_FIFO_R_DataOut: STD_LOGIC_VECTOR(7 downto 0):= (others => '0');
signal STD_FIFO_R_Empty, STD_FIFO_R_Full: STD_LOGIC:= '0';

--Declare Signals for FIFO Serial Write

signal STD_FIFO_W_WriteEn, STD_FIFO_W_ReadEn: STD_LOGIC:= '0';
signal STD_FIFO_W_DataIn, STD_FIFO_W_DataOut: STD_LOGIC_VECTOR(7 downto 0):= (others => '0');
signal STD_FIFO_W_Empty, STD_FIFO_W_Full: STD_LOGIC:= '0';

--Declare Signals for Bus Interface

signal Bus_Int1_WE, Bus_Int1_RE, Bus_Int1_Busy: STD_LOGIC:= '0';
signal Bus_Int1_DataIn, Bus_Int1_DataOut, Bus_Int1_AddrIn:
STD_LOGIC_VECTOR(15 downto 0):= (others => '0');

--Declare Signals for Registers

signal LD_Busy, LD_busy2, LD_rx, LD_tx, LD_temp_data, LD_temp2: STD_LOGIC:= '0';
signal LD_Temp_Addr_High, LD_Temp_addr_Low, LD_Temp_Data_High: STD_LOGIC:=

```

'0';
signal LD_Temp_Data_Low, ld_temp_cmd: STD_LOGIC:= '0';
signal Ld_Pkt_Len, LD_Chk_Sum: STD_LOGIC:= '0';
signal LD_Base_Addr: STD_LOGIC:= '0';
signal LD_Reg_Addr_H, LD_Reg_Addr_L: STD_LOGIC:= '0';
signal LD_Reg_Addr: STD_LOGIC:= '0';
signal LD_Reg_Cnt: STD_LOGIC:= '0';
signal LD_Data_Temp_H, LD_Data_Temp_L: STD_LOGIC:= '0';

signal busy, busy_reg_o, busy2, busy2_reg_o, rx, rx_reg_o, tx, tx_reg_o:
STD_LOGIC:= '0';
signal temp_data_reg_o, temp_data: std_logic_vector(15 downto 0):= (others =>
'0');
signal temp2_reg_o, temp2: STD_LOGIC_VECTOR(7 downto 0):= (others => '0');
signal Temp_Addr_High_reg_o, Temp_Addr_High: std_logic_vector(7 downto 0):=
(others => '0');
signal Temp_Addr_Low_reg_o, Temp_Addr_Low: std_logic_vector(7 downto 0):=
(others => '0');
signal Temp_Data_High_reg_o, Temp_Data_High: std_logic_vector(7 downto
0):=(others => '0');
signal Temp_Data_Low_reg_o, Temp_Data_Low: std_logic_vector(7 downto
0):=(others => '0');
signal Temp_Cmd_Reg_o, Temp_Cmd: std_logic_vector(7 downto 0):= (others =>
'0');
signal Pkt_Len_reg_o, Pkt_Len: STD_LOGIC_VECTOR(7 downto 0):= (others=> '0');
signal Chk_Sum_reg_o, Chk_Sum: std_logic_vector(15 downto 0):= (others =>
'0');
signal Base_Addr_reg_o, Base_Addr: std_logic_vector(15 downto 0):= (others =>
'0');
signal Reg_Addr_H_reg_o, Reg_Addr_H: STD_LOGIC_VECTOR(7 downto 0):= (others
=> '0');
signal Reg_Addr_L_reg_o, Reg_Addr_L: std_logic_vector(7 downto 0):= (others
=> '0');
signal Reg_Addr_reg_o, Reg_Addr: STD_LOGIC_VECTOR(15 downto 0):= (others =>
'0');
signal Reg_Cnt_reg_o, reg_cnt: STD_LOGIC_VECTOR(7 downto 0):= (others =>
'0');
signal Data_Temp_H_reg_o, Data_Temp_H: STD_LOGIC_VECTOR(7 downto 0):= (others
=> '0');
signal Data_Temp_L_reg_o, Data_Temp_L: STD_LOGIC_VECTOR(7 downto 0):= (others
=> '0');

--Signals for Counters

```



```

signal Rcv_Cnt_rst, Rcv_Cnt_INC: STD_LOGIC:= '0';
signal Rcv_Cnt_Out: std_logic_vector(7 downto 0):= (others => '0');
signal Buf_Cnt_rst, Buf_Cnt_INC: STD_LOGIC:= '0';
signal Buf_Cnt_Out: std_logic_vector(7 downto 0):= (others => '0');
signal Reg_Cnt_rst, Reg_Cnt_INC: STD_LOGIC:= '0';
signal Reg_Cnt_Out: std_logic_vector(7 downto 0):= (others => '0');

--signals for RAM (Storing temp packet Data) [256 Max]
type ram_type is array (0 to (2**8)-1) of std_logic_vector(7 downto 0);
signal RAM : ram_type;
signal RAM_wea: STD_LOGIC:= '0';
signal RAM_address, RAM_Data_In, RAM_Data_Out : std_logic_vector(7 downto 0);

---User defined variables
-- CM is the clock Divider 25 MHz/Cm=115,200 Baud
constant CM : integer := clk_in / Baud;
--CN is the read offset for serial input
constant CN : integer := CM/2;
---End User defined Variables

--DECLARE STD_FIFO
COMPONENT STD_FIFO
Generic (
    DATA_WIDTH : integer; --Width of FIFO
    FIFO_DEPTH : integer; --Depth of FIFO
    FIFO_ADDR_LEN : integer --Required number of bits to represent FIFO_Depth
);
Port (
    CLK : in STD_LOGIC;

    RST : in STD_LOGIC;
    WriteEn : in STD_LOGIC;
    DataIn : in STD_LOGIC_Vector (7 downto 0);
    ReadEn : in STD_LOGIC;
    DataOut : out STD_LOGIC_VECTOR (7 downto 0);
    Empty : out STD_LOGIC;
    Full : out STD_LOGIC
);
end component;

--declare Bus Interface
COMPONENT Bus_Int
Port(
    clk          :    IN   std_logic;

```

```

    rst      :    IN  std_logic;
    DataIn   :    IN  std_logic_vector(15 downto 0);
    DataOut  :    OUT std_logic_vector(15 downto 0);
    AddrIn   :    IN  std_logic_vector(15 downto 0);
    WE       :    IN   std_logic;
    RE       :    IN   std_logic;
    Busy     :    OUT std_logic;
    Data     :    INOUT std_logic_vector(15 downto 0);
    Addr     :    OUT std_logic_vector(15 downto 0);
    Xrqst    :    OUT   std_logic;
    XDat     :    IN   std_logic;
    YDat     :    OUT   std_logic;
    BusRqst  :    OUT   std_logic;
    BusCtrl  :    IN   std_logic
);
END COMPONENT;

```

```

---declare STD_Counter COMPONENT
component STD_Counter is
Generic
(
    Width : integer    --width of Counters
);
port(INC, rst, clk: in std_logic;
      Count: out STD_LOGIC_Vector(Width-1 downto 0));
end component;

```

begin

```

---Instantiate STD_FIFO for Reading Serial Data
STD_FIFO_R: STD_FIFO
Generic map
(
    DATA_WIDTH => 8,      --- Width of FIFO
    FIFO_DEPTH => 512,    --- Depth of FIFO
    FIFO_ADDR_LEN => 9    --- Required number of bits to represent FIFO_Depth
)
Port Map
(
    CLK => clk,
    RST => rst,
    WriteEn => STD_FIFO_R_WriteEn,
    DataIn  => STD_FIFO_R_DataIn,

```

```

    ReadEn      => STD_FIFO_R_ReadEn,
    DataOut     => STD_FIFO_R_DataOut,
    Empty       => STD_FIFO_R_Empty,
    Full        => STD_FIFO_R_Full
);

--Instantiate STD_FIFO for Writing Serial Data
STD_FIFO_W: STD_FIFO
Generic Map
(
    DATA_Width => 8,
    FIFO_DEPTH  => 512,
    FIFO_ADDR_LEN => 9
)
Port Map(
    CLK => clk,
    RST => rst,
    WriteEn => STD_FIFO_W_WriteEn,
    DataIn  => STD_FIFO_W_DataIn,
    ReadEn  => STD_FIFO_W_ReadEn,
    DataOut => STD_FIFO_W_DataOut,
    Empty   => STD_FIFO_W_Empty,
    Full    => STD_FIFO_W_Full
);

---Instantiate Bus Interface
Bus_Int1: Bus_Int Port MAP (
    clk => clk,
    rst => rst,
    DataIn => Bus_Int1_DataIn,
    DataOut => Bus_Int1_DataOut,
    AddrIn => Bus_Int1_AddrIn,
    WE => Bus_Int1_WE,
    RE => Bus_Int1_RE,
    Busy => Bus_Int1_Busy,
    Data => Data,
    Addr => Addr,
    Xrqst => Xrqst,
    XDat => XDat,
    YDat => YDat,
    BusRqst => BusRqst,
    BusCtrl => BusCtrl
);

```

```

--instantiate Rcv_Cnt_8
Rcv_Cnt: STD_Counter
generic Map
(
    Width => 8
)
port map(
    clk      =>  clk,
    rst      =>  Rcv_Cnt_rst,
    INC      =>  Rcv_Cnt_INC,
    Count    =>  Rcv_Cnt_Out
);

--instantiate Buf_Cnt_8
Buf_Cnt: STD_Counter
generic Map
(
    Width => 8
)
port map(
    clk => clk,
    rst => Buf_Cnt_rst,
    INC => Buf_Cnt_INC,
    Count => Buf_Cnt_Out
);

--instantiate Reg_Cnt_8
Reg_Cnt1: STD_Counter
generic Map
(
    Width => 8
)
port map(
    clk => clk,
    rst => Reg_Cnt_Rst,
    INC => Reg_Cnt_INC,
    Count => Reg_Cnt_Out);
--Registers
Reg_Proc: process
begin
    wait until clk'event and clk ='1';
    if rst = '0' then

```

```

    busy_reg_o <= '0';
    busy2_reg_o <= '0';
    rx_reg_o <= '0';
    tx_reg_o <= '0';
    temp_data_reg_o <= (others => '0');
    temp2_reg_o <= (others => '0');
    Temp_Addr_High_reg_o <= (others => '0');
    Temp_Addr_Low_reg_o <= (others => '0');
    Temp_Data_High_reg_o <= (others => '0');
    Temp_Data_Low_reg_o <= (others => '0');
    Temp_Cmd_Reg_o <= (others => '0');
    Pkt_Len_reg_o <= (others => '0');
    Chk_Sum_reg_o <= (others => '0');
    Reg_Addr_H_reg_o <= (others => '0');
    Reg_Addr_L_reg_o <= (others => '0');
    Reg_Addr_reg_o <= (others => '0');
    Data_Temp_H_reg_o <= (others => '0');
    Data_Temp_L_reg_o <= (others => '0');
    Reg_Cnt_reg_o <= (others => '0');
    Base_Addr_reg_o <= (others => '0');

else
    if (LD_Busy = '1') then busy_reg_o <= busy; end if;
    if (LD_busy2 = '1') then busy2_reg_o <= busy2; end if;
    if (LD_rx = '1') then rx_reg_o <= rx; end if;
    if (LD_tx = '1') then tx_reg_o <= tx; end if;
    if (LD_temp_data = '1') then temp_data_reg_o <= temp_data; end if;
    if (LD_temp2 = '1') then temp2_reg_o <= temp2; end if;
    if (LD_Temp_Addr_High = '1') then Temp_Addr_High_reg_o <= Temp_Addr_High;
end if;
    if (LD_Temp_addr_Low = '1') then Temp_Addr_Low_reg_o <= Temp_Addr_Low;
end if;
    if (LD_Temp_Data_High = '1') then Temp_Data_High_reg_o <= Temp_Data_High;
end if;
    if (LD_Temp_Data_Low = '1') then Temp_Data_Low_reg_o <= Temp_Data_Low;
end if;
    if (ld_temp_cmd = '1') then Temp_Cmd_Reg_o <= Temp_Cmd; end if;
    if (Ld_Pkt_Len = '1') then Pkt_Len_reg_o <= Pkt_Len; end if;
    if (LD_Chk_Sum = '1') then Chk_Sum_reg_o <= Chk_Sum; end if;
    if (LD_Reg_Addr_H = '1') then Reg_Addr_H_reg_o <= Reg_Addr_H; end if;
    if (LD_Reg_Addr_L = '1') then Reg_Addr_L_reg_o <= Reg_Addr_L; end if;
    if (LD_Reg_Addr = '1') then Reg_Addr_reg_o <= Reg_Addr; end if;
    if (LD_Data_Temp_H = '1') then Data_Temp_H_reg_o <= Data_Temp_H; end if;

```

```

        if (LD_Data_Temp_L = '1') then Data_Temp_L_reg_o <= Data_Temp_L; end if;
        if (LD_Reg_Cnt = '1') then Reg_Cnt_reg_o <= reg_cnt; end if;
        if (LD_Base_Addr = '1') then Base_Addr_reg_o <= Base_Addr; end if;
    end if;
end process;
-----End Registers

-----Process Memory Data_Reg: Process
Ram_Proc: Process
begin
    wait until clk'event and clk = '1';

    if (RAM_wea = '1') then
        RAM(conv_integer(RAM_address)) <= RAM_Data_In;
    end if;
    RAM_Data_Out <= RAM(conv_integer(RAM_address));
end process Ram_Proc;
-- End RAM

-- Next State Logic for Serial Interface read
NSL_RS232_R: process(CS_RS232_R, rs232_rcv_s, rx_done, STD_FIFO_R_Full,
temp_rcv)
begin

    -----Default States to remove latches
    busy <= '0';
    rx <= '0';
    NS_RS232_R <= S0;
    LD_Busy <= '0';
    LD_rx <= '0';

    --Signals for FIFO
    STD_FIFO_R_WriteEn <= '0';
    STD_FIFO_R_DataIn <= (others => '0');

    case CS_RS232_R is
        when S0 =>
            if (rs232_rcv_s = '1') then
                NS_RS232_R <= S0;
            else
                NS_RS232_R <= S1;
            end if;

```

```

        busy <= '0';
        rx <= '0';
        LD_rx <= '1';
        LD_Busy <= '1';

    when S1 =>
        NS_RS232_R<=S2;
        busy <= '1';
        rx <= '1';
        LD_rx <= '1';
        LD_Busy <= '1';

    when S2 =>
        if (rx_done = '0') then
            NS_RS232_R <= S2;
        else
            NS_RS232_R <= S3;
        end if;

    when S3 =>
        if (STD_FIFO_R_Full = '0') then
            STD_FIFO_R_DataIn <= temp_rcv;
            STD_FIFO_R_WriteEn <= '1';
        end if;
        NS_RS232_R <= S0;

    when others =>
        NS_RS232_R <= S0;

    end case;
end process;
-- End Next State Logic for Serial Interface Read

-- Next State Logic for Serial Interface Write
NSL_RS232_W: process(CS_RS232_W, tx_done, STD_FIFO_W_Empty,
STD_FIFO_W_DataOut)
begin

    ----Default States to remove latches
    tx <= '0';
    NS_RS232_W <= S0;
    temp2 <= (others => '0');
    LD_tx <= '0';

```

```

LD_temp2 <= '0';
Busy2 <= '0';
LD_busy2 <= '0';

--Signals for FIFO
STD_FIFO_W_ReadEn <= '0';

case CS_RS232_W is

  when S0 =>
    if(STD_FIFO_W_Empty = '1') then
      NS_RS232_W<=S0;
    else
      NS_RS232_W<=S1;
      STD_FIFO_W_ReadEn <= '1';
    end if;
    busy2 <= '0'; -- the busy signal stops the baud generator
    tx <= '0';      -- the signals to stop sending Data
    LD_tx <= '1';
    LD_busy2 <= '1';

  when S1 =>
    temp2<= STD_FIFO_W_DataOut;
    LD_temp2<='1';
    NS_RS232_W<=S2;

  when S2=>
    busy2 <='1';
    tx<='1';
    LD_tx<='1';
    LD_busy2<='1';
    NS_RS232_W<=S3;

  when S3 =>
    if(tx_done='0') then
      NS_RS232_W <= S3;
    else
      NS_RS232_W <= S0;
    end if;

  when others =>
    NS_RS232_W <= S0;
end case;

```



```

end process;
----End Next State Logic for Serial Interface Write

----Next State Logic for FIFO to bus
NSL_FIFO_Bus: process(CS_FIFO_Bus,
STD_FIFO_R_Empty,Temp_Cmd_reg_o,Bus_Int1_Busy,STD_FIFO_R_DataOut,Temp_Addr_Hi
gh_reg_o,Temp_Addr_Low_reg_o,Temp_Data_High_reg_o,Temp_Data_Low_reg_o,Temp_Da
ta_reg_o,Bus_Int1_DataOut,temp_data_reg_o,Pkt_Len_reg_o,Chk_Sum_reg_o,Rcv_Cnt
_Out,RAM_Data_Out,Reg_Cnt_reg_o,Reg_Addr_H_reg_o,Reg_Addr_L_reg_o,Base_Addr_r
eg_o )
begin

    ----Default States to remove latches
    NS_FIFO_Bus <= S0;
    Temp_Cmd <= (others => '0');
    ld_temp_cmd <= '0';
    Temp_Addr_High <= (others => '0');
    LD_Temp_Addr_High <= '0';
    Temp_Addr_Low <= (others => '0');
    LD_Temp_Addr_Low <= '0';
    Bus_Int1_AddrIn <= (others => '0');
    Bus_Int1_RE <= '0';
    Bus_Int1_DataIn <= (others => '0');
    Bus_Int1_WE <= '0';
    Temp_Data <= (others => '0');
    LD_temp_data <= '0';
    Temp_Data_High <= (others => '0');
    LD_Temp_Data_High <= '0';
    Temp_Data_Low <= (others => '0');
    LD_Temp_Data_Low <= '0';

    --Signals for FIFO
    STD_FIFO_R_ReadEn <= '0';
    STD_FIFO_W_DataIn <= (others => '0');
    STD_FIFO_W_WriteEn <= '0';

    --Signals for Counters
    Rcv_Cnt_rst <= '1';
    Rcv_Cnt_INC <= '0';
    Buf_Cnt_rst <= '1';
    Buf_Cnt_INC <= '0';
    Reg_Cnt_rst <= '1';

```

```

Reg_Cnt_INC <= '0';

--Signals for memory
RAM_address <= (others => '0');
RAM_Data_In <= (others => '0');
RAM_wea <= '0';

Pkt_Len <= (others => '0');
Ld_Pkt_Len <= '0';
Chk_Sum <= (others => '0');
LD_Chk_Sum <= '0';

Reg_Addr_H <= (others => '0');
LD_Reg_Addr_H <= '0';
Reg_Addr_L <= (others => '0');
LD_Reg_Addr_L <= '0';
Reg_Addr <= (others => '0');
LD_Reg_Addr <= '0';
Data_Temp_H <= (others => '0');
LD_Data_Temp_H <= '0';
Data_Temp_L <= (others => '0');
LD_Data_Temp_L <= '0';
reg_cnt <= (others => '0');
LD_Reg_Cnt <= '0';
Base_Addr <= (others => '0');
LD_Base_Addr <= '0';

case CS_FIFO_Bus is

    when S0 =>
        if(STD_FIFO_R_Empty = '1') then
            NS_FIFO_Bus <= S0;
        else
            NS_FIFO_Bus <= S1;
            STD_FIFO_R_ReadEn <= '1';
        end if;
        Rcv_Cnt_rst <= '0';
        Buf_Cnt_rst <= '0';
        Reg_Cnt_rst <= '0';

    when S1 =>
        Temp_Cmd <= STD_FIFO_R_DataOut;
        ld_temp_cmd <= '1';

```

```

NS_FIFO_Bus <= S2;

when S2 =>
    if(Temp_Cmd_Reg_o = X"7E") then
        NS_FIFO_Bus <= S3;
    else
        NS_FIFO_Bus<=S0;
    end if;

when S3 =>
    if(STD_FIFO_R_Empty = '1') then
        NS_FIFO_Bus <= S3;
    else
        NS_FIFO_Bus <= S4;
        STD_FIFO_R_ReadEn <= '1';
    end if;

when S4 =>
    Pkt_Len<= STD_FIFO_R_DataOut;
    Ld_Pkt_Len<= '1';
    NS_FIFO_Bus<=S5;

when S5 =>
    if((Pkt_Len_reg_o < X"FF") and (Pkt_Len_reg_o > X"03")) then
        NS_FIFO_Bus <= S6;
    else
        NS_FIFO_Bus <= S0;
    end if;
    Chk_Sum <= X"0000";
    LD_Chk_Sum <= '1';
    Rcv_Cnt_rst <= '0';

when S6 =>
    if(Rcv_Cnt_Out < (Pkt_Len_reg_o+1)) then
        NS_FIFO_Bus <= S7;
    else
        NS_FIFO_Bus <= S9;
    end if;

when S7=>
    if(STD_FIFO_R_Empty = '1') then
        NS_FIFO_Bus <= S7;
    else

```

```

        NS_FIFO_Bus<= S8; -----change back to S8
        STD_FIFO_R_ReadEn <= '1';
    end if;

when S8 =>
    if(Rcv_Cnt_Out < (Pkt_Len_reg_o )) then    -----Added the -1 4/6
        LD_Chk_Sum <= '1';
    end if;

    RAM_Data_In <= STD_FIFO_R_DataOut;
    RAM_address <= Rcv_Cnt_Out;
    RAM_wea <= '1';
    Chk_Sum <= Chk_Sum_reg_o + STD_FIFO_R_DataOut;
    Temp_Cmd <= STD_FIFO_R_DataOut;
    ld_temp_cmd <= '1';
    Rcv_Cnt_INC <= '1';
    NS_FIFO_Bus <= S6;

when S9=>
    if((X"FF"-Chk_Sum_reg_o(7 downto 0)) = Temp_Cmd_Reg_o) then
        NS_FIFO_Bus <= S10;
    else
        NS_FIFO_Bus <= S0; -----
-----Change back to S0
    end if;
    Rcv_Cnt_rst <= '0';
    RAM_address <= X"01";

when S10 =>
    reg_cnt <= RAM_Data_Out;
    LD_Reg_Cnt <= '1';
    RAM_address <= X"01";
    NS_FIFO_Bus <= S11;

when S11 =>
    RAM_address <= X"02";
    NS_FIFO_Bus <= S12;

when S12 =>
    Reg_Addr_H <= RAM_Data_Out;
    LD_Reg_Addr_H <= '1';
    RAM_address <= X"02";
    NS_FIFO_Bus <= S13;

```

```

when S13 =>
    RAM_address <= X"03";
    NS_FIFO_Bus <= S14;

when S14 =>
    Reg_Addr_L <= RAM_Data_Out;
    LD_Reg_Addr_L <= '1';
    RAM_address <= X"03";
    NS_FIFO_Bus <= S15;

when S15 =>
    Base_Addr(15 downto 8) <= Reg_Addr_H_reg_o;
    Base_Addr(7 downto 0) <= Reg_Addr_L_reg_o;
    LD_Base_Addr <= '1';
    RAM_address <= X"00";
    NS_FIFO_Bus <= S16;

when S16 =>
    if(RAM_Data_Out = X"0F") then          -- Check CMD (Head)
        NS_FIFO_Bus <= S17;
    elsif(RAM_Data_Out = X"0A") then      -- Check CMD (Write)
        NS_FIFO_Bus <= S28;
    else
        NS_FIFO_Bus <= S0;                -- Check CMD (Invalid
Data)
    end if;
    RAM_address <= X"00";
    Rcv_Cnt_rst<= '0';

    --start Read command sequence
when S17 =>
    STD_FIFO_W_DataIn <= X"7E";
    STD_FIFO_W_WriteEn <= '1';
    Chk_Sum <= X"00FF";
    LD_Chk_Sum <= '1';
    NS_FIFO_Bus<= S18;

when S18 =>
    STD_FIFO_W_DataIn <= ((Reg_Cnt_reg_o(7 downto 0) +
Reg_Cnt_reg_o(7 downto 0)) + 4);
    STD_FIFO_W_WriteEn <= '1';
    NS_FIFO_Bus <= S19;

```

```

when S19 =>
    STD_FIFO_W_DataIn <= X"0A";
    STD_FIFO_W_WriteEn <= '1';
    Chk_Sum <= Chk_Sum_reg_o - X"0A";
    LD_Chk_Sum <= '1';
    NS_FIFO_Bus <= S20;

when S20 =>
    STD_FIFO_W_DataIn <= Reg_Cnt_reg_o;
    STD_FIFO_W_WriteEn <= '1';
    Chk_Sum <= Chk_Sum_reg_o - Reg_Cnt_reg_o;
    LD_Chk_Sum <= '1';
    NS_FIFO_Bus <= S21;

when S21 =>
    STD_FIFO_W_DataIn <= Reg_Addr_H_reg_o;
    STD_FIFO_W_WriteEn <= '1';
    Chk_Sum <= Chk_Sum_reg_o - Reg_Addr_H_reg_o;
    LD_Chk_Sum <= '1';
    NS_FIFO_Bus <= S22;

when S22 =>
    STD_FIFO_W_DataIn <= Reg_Addr_L_reg_o;
    STD_FIFO_W_WriteEn <= '1';
    Chk_Sum <= Chk_Sum_reg_o - Reg_Addr_L_reg_o;
    LD_Chk_Sum <= '1';
    NS_FIFO_Bus <= S23;

when S23 =>
    if(Rcv_Cnt_Out < Reg_Cnt_reg_o) then
        Bus_Int1_AddrIn <= Base_Addr_reg_o + Rcv_Cnt_Out;
        Bus_Int1_RE <= '1';
        NS_FIFO_Bus <= S24;
    else
        NS_FIFO_Bus <= S27;
    end if;

when S24 =>
    if(Bus_Int1_Busy = '1') then
        NS_FIFO_Bus <= S24;
    else
        NS_FIFO_Bus <= S25;
    end if;

```

```

        end if;
        Temp_Data <= Bus_Int1_DataOut;
        LD_temp_data <= '1';

when S25 =>
    STD_FIFO_W_DataIn <= temp_data_reg_o(15 downto 8);
    STD_FIFO_W_WriteEn <= '1';
    Chk_Sum <= Chk_Sum_reg_o - temp_data_reg_o(15 downto 8);
    LD_Chk_Sum <= '1';
    NS_FIFO_Bus <= S26;

when S26 =>
    STD_FIFO_W_DataIn <= temp_data_reg_o(7 downto 0);
    STD_FIFO_W_WriteEn <= '1';
    Chk_Sum <= Chk_Sum_reg_o - temp_data_reg_o(7 downto 0);
    LD_Chk_Sum <= '1';
    Rcv_Cnt_INC <= '1';
    NS_FIFO_Bus <= S23;

when S27 =>
    STD_FIFO_W_DataIn <= Chk_Sum_reg_o(7 downto 0);
    STD_FIFO_W_WriteEn <= '1';
    NS_FIFO_Bus <= S0;
--End Read command sequence

--Start Write Command sequence
when S28 =>
    if(Rcv_Cnt_Out < Reg_Cnt_reg_o) then
        NS_FIFO_Bus <= S29;
    else
        NS_FIFO_Bus <= S0;
    end if;
    RAM_address <= Rcv_Cnt_Out + Rcv_Cnt_Out + 4;

when S29 =>
    RAM_address <= Rcv_Cnt_Out + Rcv_Cnt_Out + 4;
    Temp_Data_High <= RAM_Data_Out;
    --LD_Temp_Data_High <= '1';
    NS_FIFO_Bus <= S30;

when S30 =>
    RAM_address <= Rcv_Cnt_Out + Rcv_Cnt_Out + 4;
    Temp_Data_High <= RAM_Data_Out;

```

```

        LD_Temp_Data_High <= '1';
        NS_FIFO_Bus <= S31;

    when S31 =>
        RAM_address <= Rcv_Cnt_Out + Rcv_Cnt_Out + 5;
        NS_FIFO_Bus <= S32;

    when S32 =>
        RAM_address <= Rcv_Cnt_Out + Rcv_Cnt_Out + 5;
        Temp_Data_Low <= RAM_Data_Out;
        -- LD_Temp_Data_Low <= '1';
        NS_FIFO_Bus <= S33;

    when S33 =>
        RAM_address <= Rcv_Cnt_Out + Rcv_Cnt_Out + 5;
        Temp_Data_Low <= RAM_Data_Out;
        LD_Temp_Data_Low <= '1';
        NS_FIFO_Bus <= S34;

    when S34=>
        NS_FIFO_Bus <= S35;

    when S35 =>
        Bus_Int1_AddrIn <= Base_Addr_reg_o + Rcv_Cnt_Out;
        Bus_Int1_DataIn(15 downto 8) <= Temp_Data_High_reg_o;
        Bus_Int1_DataIn(7 downto 0) <= Temp_Data_Low_reg_o;
        Bus_Int1_WE <= '1';
        NS_FIFO_Bus <= S36;

    when S36 =>
        if( Bus_Int1_Busy = '1') then
            NS_FIFO_Bus <= S36;
        else
            Rcv_Cnt_INC <= '1';
            NS_FIFO_Bus <= S28;
        end if;
    -- End Write Command sequence

    when others =>
        NS_FIFO_Bus <= S0;

end case;
end process;

```


----End Next State Logic for FIFO to Bus

----UART Clock Divider

UART_Clk: Process

begin

wait until clk'event and clk = '1';

--Synch async signal

rs232_rcv_t <= rs232_rcv; --Synchro1 rs232_rcv

rs232_rcv_s <= rs232_rcv_t;

if(rst = '0' or (busy_reg_o = '0' and busy2_reg_o = '0')) then

uartclk <= '0';

i <= CONV_STD_LOGIC_VECTOR(CN,16);

elsif (i = CM) then

uartclk <= '1';

i <= X"0000";

else

i <= i+1;

uartclk <= '0';

end if;

end Process;

--END UART Clock Divider

---UART_READ

UART_READ: Process

begin

wait until clk'event and clk = '1';

if rst = '0' or rx_reg_o = '0' then

temp_rcv <= x"00";

j <= x"0000";

rx_done <= '0';

elsif rx_reg_o = '1' then

if uartclk = '1' then

if j < X"09" then

temp_rcv(7) <= rs232_rcv_s;

temp_rcv(6 downto 0) <= temp_rcv(7 downto 1);

j <= j+1;

rx_done <= '0';

else

j <= X"0000";

rx_done <= '1';

end if;

else

```

        rx_done <= '0';
    end if;
end if;
end process;
-- End UART read

--UART Xmit
UART_Xmit: Process
begin
    wait until clk'event and clk = '1';
    if (rst = '0' or tx_reg_o = '0') then
        rs232_xmt <= '1';
        tx_done <= '0';
        u <= 0;

        --structure the 10-bit frame to be sent
        txbuff(9) <= '1'; --stopbit 2
        txbuff(8 downto 1) <= temp2_reg_o;
        txbuff(0) <= '0';
    else
        if uartclk = '1' then
            if(u<10) then
                rs232_xmt <= txbuff(0);
                txbuff(8 downto 0) <= txbuff(9 downto 1);
                tx_done <= '0';
                u <= u +1;
            else
                u <= 0;
                tx_done <= '1';
            end if;
        end if;
    end if;
end process;
---End UART_Xmit

--State Synch
Sync_States: Process
begin
    wait until clk'event and clk = '1';
    if rst = '0' then
        CS_RS232_R <= S0;
        CS_RS232_W <= S0;
        CS_FIFO_Bus <= S0;
    end if;
end process;

```

```
    else
        CS_RS232_R <= NS_RS232_R;
        CS_RS232_W <= NS_RS232_W;
        CS_FIFO_Bus <= NS_FIFO_Bus;
    end if;
end process;
--End State Sync

end Behavioral;
```

Appendix B (Bus Interface Top):

-- Company: University of Arkansas (NCREPT) -- Engineer: Tyler Cash -- -- Create Date: 25March2026 --
Last Updated: 7AprilMar2026 -- Design Name: Bus Interface w/ UART -- Module Name:
BusInterface_UART - Behavioral -- Project Name: BusInterface_UART -- Target Devices: LCMXO3D-
9400HC-6BG256C (MachXO3D_BreakoutBrd) -- Tool versions: Lattice Diamond_x64 Build 3.12.0.240.2 --
Description: -- This Project was developed to demonstrate a system-level design which incorporates
shared registers and communications. -- You may use various Serial Interface programs to communicate
with this including the old version of X-CTU (Version 5.2.8.6), -- Termite, HypterTerminal, or the Custom
LabVIEW Interface created for this project (LabVIEW-CommEx_Bus_Interface_PWM_v1.0a_16Apr2021).
-- A key to remember is that you will need to create packets in Hex-Mode not the standard ASCII-Mode.
-- A Testbench for this project was created and demonstrates base functionality. -- ---- PinOut: -- Signal
CPLD_Pin Description -- SCI_TX C11 Serial-TX (Output) [USB Serial Port on Breakout Board] -- SCI_RX A11
Serial-RX (Input) [USB Serial Port on Breakout Board] -- ---- Register and Memory Map Information: --
This section describes the Memory Map used in this project. -- This design contains a SPRAM Module
which is 16 bits wide and 1024 entries deep. -- Register addresses are from X"0000" to X"03FF". -- All
registers are 16-bits wide. -- The SPRAM Module is located in the Bus_Master portion of the code. -- This
RAM Module may be accessed externally using Serial Port interface. -- Reserved for future use. --
X"0300" - X"03FF" -- Register Map is found as constants in Bus_Interface_Common and shared with all
submodules of this program. -- ---- Serial Settings -- Baud: 9600 -- Flow Control: None -- Data Bits: 8 --
Parity: None -- Stop Bits: 1 -- -- -- ---- Serial Communications: -- In the project we will construct packets
to facilitate serial communication between the device and a computer. -- Using packets we can
implement operational commands and check sums which allow for easily implementing a user GUI. --
You may use various Serial Interface programs to communicate with this including the old version of X-
CTU (Version 5.2.8.6), -- Termite, HypterTerminal, or the Custom LabVIEW Interface created for this
project (LabVIEW-CommEx_Bus_Interface_PWM_v1.0a_16Apr2021) -- A key to remember is that you
will need to create packets in Hex-Mode not the standard ASCII-Mode. -- All packets will begin with a
Start Delimiter (0x7E) and end with a CheckSum. -- The CheckSum is calculated by adding up all the
bytes of the packet between the packet length and the checksum then -- subtracting the sum from 0xFF.
-- We will implement two types of packets for our use. -- Register Write Command Packet (Used to
update registers internal to the CPLD) -- Register Read Command Packet (Used to read registers internal
to the CPLD) -- ---Serial Write Packet (OP_ID = 0x0A) -- The Register Write Command Packet is used to
update registers internal to the CPLD. -- The following example breaks down a write request packet. --
The packet below writes 7 16-bit registers starting at register 0x0100; Values listed below. --
PWM_Enable => 0x0001 (1=Enable; 0=Disable) -- LED_BlinkFreq => 0xBFFF [75%] (BFFF/FFFF) (%) --
LED_OnTime => 0x6000 [50%] (6000/BFFF) (%) -- LED1_Intensity => 0x2000 [12.5%] (2000/FFFF) (%) --
LED2_Intensity => 0x4000 [25%] (4000/FFFF) (%) -- LED3_Intensity => 0x8000 [50%] (8000/FFFF) (%) --
LED4_Intensity => 0xFFFF [100%] (FFFF/FFFF) (%) -- -- Start Delimiter | Pkt Len | Op ID | Register_Cnt
| Start Address | Register Data (16bit x Register_Cnt) | ChkSum -- 0x7E | 0x12 | 0x0A | 0x07 | 0x0100
| 0x0001BFFF6000200040008000FFFF | 0xF0 -- 0x7E120A0701000001BFFF6000200040008000FFFFF0

results in all LEDs being set to the above parameters. --
 0x7E120A0701000001BFFF60000000000000000000CE results in all LED Intensities being set to 0%. -- ---
 Serial Read Packet (OP_ID = 0x0F) -- The Register Read Command Packet is used to read registers
 internal to the CPLD. -- The following example breaks down a read request packet. -- The packet below
 reads 16 16-bit registers starting at register 0x0100. -- Start Delimiter| Pkt Len |Op ID| Register_Cnt
 |Start Address | ChkSum -- 0x7E | 0x04 |0x0F |0x10 |0x0100 | 0xDF -- 0x7E040F100100DF -- -- The
 above command results in a write command being sent from the CPLD which contains data from 16 16-
 bit registers starting at address 0x0100. -- An example response from the CPLD is shown below: --
 0x7E240A1001000001BFFF6000200040008000FFFF000E7 --
 Start Delimiter| Pkt Len |Op ID| Register_Cnt |Start Address | Register Data (16bit x Register_Cnt) |
 ChkSum

-- 0x7E | 0x24 |0x0A |0x10 |0x0100
 |0x0001BFFF6000200040008000FFFF00
 000000 | 0xE7

--

--

-- Revisions:--

---- Revision 3.0a --- Updated to just include UART Interface for class to ease
 knowledge transitions. ---- Revision 2.2c --- Updated back to SPRAM Module
 for testbench compatibility. -- Updated on 21Apr2021 to include correct pin
 numbers in documentation and four PWM modules designed in class. -- Added
 design files and drawings for PWMs to Supplemental Folder. -- ---- Revision
 2.2b --- Updated back to SPRAM Module for testbench compatibility. --
 Updated on 8Apr2021 so it is compatible with the testbench, will update again
 in the future back to DPRAM. -- ---- Revision 2.2a --- Updated from SPRAM to
 DPRAM Module -- Updated on 27Mar2021 to use a DPRAM module to take
 advantage of EBRs and maintain compatibility with other programs. -- ----
 Revision 2.1a - -- Updated for Git Implementation -- ---- Revision 2.0a - --
 Updated for MachXO3D and so it could serve as an example for the
 "Programming for Power Electronics" Class. -- Testbed updated to allow for
 Read and Write Command Verification -- Minor Updates to documentation and
 PWMs. -- Testbench updated to allow for Read and Write Command
 Verification -- ---- Revision 1.1a - -- Updated to use Protocols based on Zigbee
 Implementation -- -- Revision 1.0b - -- Updated to use UCB instead of

**Evaluation Board -- -- Revision 0.01 - -- File Created; Basic\Classical Operation
Implemented -- -- -- Additional Comments: -- --10**

```
Library IEEE;
use IEEE.std_logic_1164.all;
use ieee.numeric_std.all;
use IEEE.std_logic_arith.all;
```

```
library machxo3d;
use machxo3d.all;
```

```
library work;
use work.Bus_Interface_Common.all;
```

```
entity Bus_Interface_Top is
Port (
---- RS232 Communication
SCI_RX : in STD_LOGIC;
SCI_TX : inout STD_LOGIC
);
end Bus_Interface_Top;
```

```
architecture Behavioral of Bus_Interface_Top is
Component OSCJ
GENERIC (NOM_FREQ: string := "8.31");
PORT ( STDBY : IN std_logic;
OSC : OUT std_logic;
SEDSTDBY : OUT std_logic
);
END COMPONENT;
```

```
-- Declare PLL
Component PLLSerial
PORT (
ClkI: in std_logic;
ClkOP: out std_logic;
Lock: out std_logic
);
END COMPONENT;
```

```
-- Declare Bus Master
COMPONENT Bus_Master
PORT (
clk : IN std_logic;
rst : IN std_logic;
```

```

Data : INOUT STD_LOGIC_VECTOR(15 downto 0);
Addr : IN STD_LOGIC_VECTOR(15 downto 0);
Xrqst : IN std_logic;
XDat : OUT std_logic;
YDat : IN std_logic;
BusRqst : IN std_logic_vector(9 downto 0);
BusCtrl : OUT std_logic_vector(9 downto 0)
);
END Component;

```

```

COMPONENT RS232_Usr_Int
generic(
  Baud   : integer;
  clk_in : integer
);
Port (
  clk : in STD_LOGIC;
  rst : in STD_LOGIC;
  rs232_rcv : in STD_LOGIC;
  rs232_xmt : out STD_LOGIC;
  Data   : INOUT std_logic_vector(15 downto 0);
  Addr   : OUT std_logic_vector(15 downto 0);
  Xrqst  : OUT std_logic;
  XDat   : IN std_logic;
  YDat   : OUT std_logic;
  BusRqst : OUT std_logic;
  BusCtrl : IN std_logic
);
end Component;

```

```

-- Declare STD_Counter Component
component STD_Counter is
  GENERIC
  (
    Width : integer--width of Counters
  );
  port(INC, rst, clk: in std_logic;
  Count: out std_logic_vector(Width -1 downto 0)
  );
end component;

```

```

--signals
-- Declare Signals for Bus Interface
signal Bus_Int1_WE, Bus_Int1_RE, Bus_Int1_Busy: STD_LOGIC:='0';
signal Bus_Int1_DataIn, Bus_Int1_DataOut, Bus_Int1_AddrIn: STD_LOGIC_VECTOR(15 downto 0):=
(others => '0');

```

```

-- Inputs
signal Addr : std_logic_vector(15 downto 0):= (others => '0');
signal Xrqst : std_logic := '0';
signal YDat : std_logic := '0';
signal BusRqst : STD_LOGIC_Vector(9 downto 0):= (others => '0');
signal Data : STD_LOGIC_Vector(15 downto 0):= (others => '0');
signal XDat : STD_LOGIC := '0';
signal BusCtrl : STD_LOGIC_Vector(9 downto 0):= (others => '0');

-- internal clock
signal OSC_Stdbby, OSC_Out, OSC_SEDSTDBY, clk: std_logic := '0';

-- Reset
signal PLL_Lock, System_rst: std_logic := '0';
signal Reset_Cnt_INC, Reset_Cnt_rst: std_logic := '0';
signal Reset_Cnt_out: STD_LOGIC_VECTOR(7 downto 0):= (others => '0');

begin

-- Instantiate Internal Oscillator
Int_OSC: OSCJ Port MAP (
STDBY => OSC_Stdbby,
OSC => OSC_Out,
SEDSTDBY => OSC_SEDSTDBY
);

--Instantiate PLL
PLL_1: PLLSerial PORT MAP (
Clkl => OSC_Out,
ClkOP => clk,
Lock => PLL_Lock
);

-- Instantaite Bus_Master
BM: Bus_Master Port Map (
clk => clk,
rst => System_rst,
Data => Data,
Addr => Addr,
Xrqst => Xrqst,
XDat => XDat,
YDat => YDat,
BusRqst => BusRqst,
BusCtrl => BusCtrl
);

-- Instantaite RS232_Usr_Int

```



```

RS232_Usr: RS232_Usr_Int
Generic MAP
(
  Baud => 9600,
  Clk_in => Clk_Freq -- Input clk
)
PORT MAP (
  clk => clk,
  rst => System_rst,
  rs232_rcv => SCI_RX,
  rs232_xmt => SCI_TX,
  Data => Data,
  Addr => Addr,
  Xrqst => Xrqst,
  XDat => XDat,
  YDat => YDat,
  BusRqst => BusRqst(0),
  BusCtrl => BusCtrl(0)
);

--Instantiate Reset_Cnt_8
Reset_Cnt: STD_Counter
generic MAP
(
  Width => 8
)
port map(
  clk => OSC_Out,
  rst => Reset_Cnt_rst,
  INC => Reset_Cnt_INC,
  Count => Reset_Cnt_out
);

-- Oscillator
OSC_Stdbby <= '0';

-- Tie unused ports to '0'
BusRqst(9 downto 1) <= (others => '0');

-- Reset Block1
Reset_Blk1: Process
begin
  wait until OSC_Out'event and OSC_Out = '1';
  if(PLL_Lock = '0') then
    Reset_Cnt_rst <= '0'; -- active low
  else
    Reset_Cnt_rst <= '1';
  end if;
end if;

```

```
end process;

-- Reset Block
Reset_Blk: process
begin
wait until OSC_Out'event and OSC_Out = '1';
if(Reset_Cnt_out < X"7F") then
System_rst <= '0';
Reset_Cnt_Inc <= '1';
else
System_rst <= '1';
Reset_Cnt_Inc <= '0';
end if;
end process;

end Behavioral;
```

Appendix C (Bus_Interface_TB):

[illegible]

```
-- 0x7E | 0x24 | 0x0A | 0x10 | 0x0100
| 0x0001BFFF6000200040008000FFFF0000000000000000000000000000
000000 | 0xE7
```

..

-- Revisions:-- ---- Revision 2.0a - -- Updated for MachXO3D and so it could serve as an example for the "Programming for Power Electronics" Class. -- Testbed updated to allow for Read and Write Command Verification -- ---- Revision 1.1b - -- Minor Updates to documentation and PWMs. -- Testbed updated to allow for Read and Write Command Verification -- ---- Revision 1.1a - -- Updated to use Protocols based on Zigbee Implementation -- -- Revision 1.0b - -- Updated to use UCB instead of Evaluation Board -- -- Revision 0.01 - -- File Created; Basic\Classical Operation Implemented -- -- -- Additional Comments: -- --

```

-- LIBRARY ieee; USE ieee.std_logic_1164.ALL; USE ieee.numeric_std.ALL;
ENTITY Bus_Interface_TestBench IS END Bus_Interface_TestBench;
ARCHITECTURE behavior OF Bus_Interface_TestBench IS
COMPONENT Bus_Interface_Top
PORT(
    SCI_RX : IN std_logic;
    SCI_TX : INOUT std_logic
);
END COMPONENT;

SIGNAL SCI_RX : std_logic;
SIGNAL SCI_TX : std_logic;

-- Clock period definitions
constant clk_period : time := 20 ns;
--constant read_Time: time :=8680 ns; --for 115,200 Baud
constant read_Time: time :=104100 ns; --for 9,600 Baud

-- System Clk and Reset(Not Needed Here)
--signal SYSCLK : std_logic;
--signal RESETn : std_logic;

-- Memory Array
type Memory is array (255 downto 0) of STD_LOGIC_VECTOR (7 downto 0);
signal RS232_Cmd: Memory;

BEGIN
-- Please check and add your generic clause manually uut: Bus_Interface_Top PORT MAP( SCI_RX =>
SCI_RX, SCI_TX => SCI_TX );
---- Example Serial Commands --- Set Registers -- 7E12 0A07 0100 0001 FFFF 0000 2000 4000 8000 FFFF
10
---Read Registers -- 7E04 0F10 0100 DF
----Define Command Memory
--Test Write
RS232_Cmd(0) <= X"7E";           --Start Deliminator
RS232_Cmd(1) <= X"12";           --Pkt Length
RS232_Cmd(2) <= X"0A";           --Cmd (Read)
RS232_Cmd(3) <= X"07";           --Register Count
RS232_Cmd(4) <= X"01";           --Start Address High
RS232_Cmd(5) <= X"00";           --Start Address Low
RS232_Cmd(6) <= X"00";           --LED Enable High
RS232_Cmd(7) <= X"01";           --LED Enable Low

```

```

RS232_Cmd(8) <= X"FF";           --Blink Period High
RS232_Cmd(9) <= X"FF";           --Blink Period Low
RS232_Cmd(10) <= X"00";          --LED On Time High
RS232_Cmd(11) <= X"00";          --LED On Time Low
RS232_Cmd(12) <= X"20";          --LED 1 Intensity High
RS232_Cmd(13) <= X"00";          --LED 1 Intensity Low
RS232_Cmd(14) <= X"40";          --LED 2 Intensity High
RS232_Cmd(15) <= X"00";          --LED 2 Intensity Low
RS232_Cmd(16) <= X"80";          --LED 3 Intensity High
RS232_Cmd(17) <= X"00";          --LED 3 Intensity Low
RS232_Cmd(18) <= X"FF";          --LED 4 Intensity High
RS232_Cmd(19) <= X"FF";          --LED 4 Intensity Low
RS232_Cmd(20) <= X"10";          --Check Sum

--Test Read
RS232_Cmd(30) <= X"7E";          --Start Deliminator
RS232_Cmd(31) <= X"04";          --Pkt Length
RS232_Cmd(32) <= X"0F";          --Cmd (Read)
RS232_Cmd(33) <= X"10";          --Register Count
RS232_Cmd(34) <= X"01";          --Start Address High
RS232_Cmd(35) <= X"00";          --Start Address Low
RS232_Cmd(36) <= X"DF";          --Check Sum

-- Stimulus process stim_proc: process begin
-- initialize serial ports to idle state SCI_RX <= '1';
---- hold reset state for 100 ns.
--RESETn <='0';
--wait for clk_period*100;
--RESETn <='1';
--wait for clk_period*100;
---- insert stimulus here

for j in 0 to 20 loop
    SCI_RX <= '0';                --Send Start Bit
    wait for read_Time;
    for i in 0 to 7 loop
        SCI_RX <= RS232_Cmd(0+j)(i);
        wait for read_Time;
    end loop;
    SCI_RX <= '1';                --Send Stop Bit
    wait for read_Time;
end loop;

wait for read_Time*10;

for j in 0 to 6 loop
    SCI_RX <= '0';                --Send Start Bit

```

```

    wait for read_Time;
    for i in 0 to 7 loop
        SCI_RX <= RS232_Cmd(30+j)(i);
        wait for read_Time;
    end loop;
    SCI_RX <= '1';                                --Send Stop Bit
    wait for read_Time;
end loop;

```

```

wait; -- will wait forever

```

```

END PROCESS; END;

```